Parking Lot System

A machine-coding / LLD walkthrough — from requirements to running code

This is the classic LLD interview problem, built the way a real round expects: **clarify requirements** → **identify entities** → **build bottom-up** → **let the patterns emerge**. We never force a pattern in; each one appears naturally when the requirements call for it. Two **Strategy** patterns fall out (spot-selection and pricing), and the whole thing stays clean and un-over-engineered.

What's inside

1. How a Machine-Coding Round Flows

There's a fixed sequence. Strong candidates always follow it in order — and never jump straight to code.

1. Clarify requirements	ask questions, pin down scope — never code blind
2. Identify entities	what are the nouns? they become your classes
3. Define relationships	who owns whom, who talks to whom
4. Spot where patterns fit	ONLY after the above — patterns emerge, aren't forced
5. Code it	clean, compiling, working
6. Handle “now add X”	the interviewer extends it; your design absorbs the change

> **The two biggest mistakes:** (1) jumping straight to code without clarifying, and (2) **over-engineering** — cramming six patterns into a problem that needs two. Knowing when NOT to use a pattern is as important as knowing the pattern.

2. Step 1 — Clarifying Questions

LLD questions must be about **concrete mechanics**, not vague scope. “What features do you want?” is too high-level. Drill into the specific moving parts:

The standard question set

Multiple floors?	yes — configurable N floors
Vehicle types & spot sizes?	Bike, Car, Truck; each needs a matching spot size
How to assign a spot?	nearest / first-free / any — pluggable (Strategy!)
Entry / exit flow?	entry issues a ticket; exit computes fee & frees the spot
Pricing model?	per-hour, differs by vehicle type; keep flexible (Strategy!)
Payment methods?	cash / card / UPI (Strategy again, if extended)

> **The meta-lesson:** three of these questions — spot-assignment, pricing, payment — are secretly pointing at **Strategy** (“multiple interchangeable ways to do one thing”). We haven't written a line of code yet, and the pattern has already surfaced. **The questions ARE the design.**

3. Step 2 — Requirements (Locked Scope)

Combining the questions into a clean, complete — but not over-scoped — spec:

• N floors, configurable

• Each floor has multiple spots; each spot has a type: BIKE, CAR, TRUCK

• Each vehicle has a matching type; a vehicle can only park in a matching spot

• Entry: pick a suitable free spot (via a spot-selection strategy), park, issue a ticket with entry time

• Exit: compute fare = duration × rate (via a pricing strategy), then free the spot

• Keep spot-selection and pricing pluggable

4. Step 3 — Entities & Ownership Tree

Turn the spec's nouns into classes. Trick: circle every noun, and remember — *the thing being managed is always an entity* (that's how ParkingSpot is easy to miss).

Entity	What it is
ParkingLot	the whole system; owns floors, runs entry/exit
Floor	one level; owns spots
ParkingSpot	one space; has type + free/occupied status
Vehicle	bike / car / truck
Ticket	entry record; spot + entry time
PricingStrategy	pluggable fare calculation (emerged)
SpotSelectionStrategy	pluggable spot-picking (emerged)

The ownership tree

```
ParkingLot
   |-- owns many -> Floor
   |                 +-- owns many -> ParkingSpot
   |                                    +-- holds a -> Vehicle (when occupied)
   |-- uses a -> SpotSelectionStrategy
   |-- uses a -> PricingStrategy
   +-- issues -> Ticket
```

Read top-down: a lot has floors, a floor has spots, a spot holds a vehicle. The lot *uses* strategies and *issues* tickets. We build **bottom-up** — smallest independent bricks first, then assemble.

5. Building the Bricks (Bottom-Up)

Brick 1 — Vehicle

A vehicle's type is a **fixed set** — perfect for an `enum`. Type safety, clarity, no typos. And note: we use **one class with a type field**, not Bike/Car/Truck subclasses — because they behave identically and differ only in a value (data, not behaviour).

Vehicle.java

```
public enum VehicleType {
    BIKE, CAR, TRUCK
}

public class Vehicle {
    private String licensePlate;   // e.g. "KA01AB1234"
    private VehicleType type;      // BIKE / CAR / TRUCK

    public Vehicle(String licensePlate, VehicleType type) {
        this.licensePlate = licensePlate;
        this.type = type;
    }

    public String getLicensePlate() { return licensePlate; }
    public VehicleType getType()    { return type; }
}
```

Brick 2 — ParkingSpot

The heart of the system: the space a vehicle sits in. It **manages its own state** and owns the matching rule via `canFit()` — "am I free AND does the vehicle's type equal mine?"

ParkingSpot.java

```
public class ParkingSpot {
    private String id;             // unique: "F1-S05"
    private VehicleType spotType;  // what SIZE this spot is
    private Vehicle vehicle;       // parked here (null = empty)

    public ParkingSpot(String id, VehicleType spotType) {
        this.id = id;
        this.spotType = spotType;
        this.vehicle = null;       // starts empty
    }

    public boolean isFree() { return vehicle == null; }

    // the MATCHING RULE lives here
    public boolean canFit(Vehicle v) {
        return isFree() && v.getType() == spotType;
    }

    public void park(Vehicle v)     { this.vehicle = v; }
    public void removeVehicle()     { this.vehicle = null; }

    public String getId()           { return id; }
    public VehicleType getSpotType() { return spotType; }
    public Vehicle getVehicle()     { return vehicle; }
}
```

> **Why canFit lives here:** the spot knows its own size and status, so the “does this vehicle fit?” check belongs to it (Single Responsibility). Everyone else just asks the spot.

Brick 3 — Floor

Holds a list of spots and finds an available one — by **asking each spot** `canFit()`. It reuses the spot's logic instead of re-implementing it.

Floor.java

```
import java.util.ArrayList;
import java.util.List;

public class Floor {
    private int floorNumber;
    private List<ParkingSpot> spots;

    public Floor(int floorNumber) {
        this.floorNumber = floorNumber;
        this.spots = new ArrayList<>();
    }

    public void addSpot(ParkingSpot spot) { spots.add(spot); }

    // find a free spot that fits this vehicle
    public ParkingSpot findAvailableSpot(Vehicle vehicle) {
        for (ParkingSpot spot : spots) {
            if (spot.canFit(vehicle)) {   // reuse the spot's own logic!
                return spot;
            }
        }
        return null;   // no suitable spot on this floor
    }

    public int getFloorNumber()         { return floorNumber; }
    public List<ParkingSpot> getSpots() { return spots; }
}
```

> **The delegation chain:** Lot asks Floors, Floors ask Spots, Spots answer. Each level only knows the level directly below it — clean layered design.

Brick 4 — Ticket

The **entry↔exit link**: who parked, where, and when. It stamps its own entry time in the constructor (a ticket is created exactly at entry, so “now” is always right). Time as a `long` keeps duration = simple subtraction.

Ticket.java

```
public class Ticket {
    private String ticketId;
    private Vehicle vehicle;
    private ParkingSpot spot;
    private long entryTime;    // millis since 1970

    public Ticket(String ticketId, Vehicle vehicle, ParkingSpot spot) {
        this.ticketId = ticketId;
        this.vehicle = vehicle;
        this.spot = spot;
        this.entryTime = System.currentTimeMillis();  // stamp entry moment
    }

    public String getTicketId()  { return ticketId; }
    public Vehicle getVehicle()  { return vehicle; }
    public ParkingSpot getSpot() { return spot; }
    public long getEntryTime()   { return entryTime; }
}
```

Brick 5 — PricingStrategy (Strategy pattern #1)

Pricing is **pluggable** — the classic Strategy shape. Key insight: split by the pricing **scheme** (hourly vs daily = different behaviour = classes), and keep the per-type **rates** as data in a lookup map. Don't make BikePricing/CarPricing/TruckPricing — those differ only by a number.

PricingStrategy.java

```
import java.util.Map;

interface PricingStrategy {
    double calculate(VehicleType type, long durationHours);
}

// BEHAVIOUR: the hourly SCHEME (a class)
class HourlyPricing implements PricingStrategy {
    // DATA: per-type rates in a lookup table
    private Map<VehicleType, Double> rates = Map.of(
        VehicleType.BIKE, 10.0,
        VehicleType.CAR,  20.0,
        VehicleType.TRUCK, 40.0
    );

    public double calculate(VehicleType type, long durationHours) {
        return durationHours * rates.get(type);
    }
}

// a genuinely different SCHEME = another class
class FlatDailyPricing implements PricingStrategy {
    public double calculate(VehicleType type, long durationHours) {
        long days = (durationHours / 24) + 1;
        return days * 100.0;   // totally different formula
    }
}
```

Brick 6 — SpotSelectionStrategy (Strategy pattern #2)

How to pick a spot (nearest / first-free / random) is genuinely different logic — behaviour — so each is its own class. Adding a new strategy = a new class, never editing existing ones (Open/Closed).

SpotSelectionStrategy.java

```
import java.util.List;

interface SpotSelectionStrategy {
    ParkingSpot selectSpot(List<Floor> floors, Vehicle vehicle);
}

class FirstAvailableStrategy implements SpotSelectionStrategy {
    public ParkingSpot selectSpot(List<Floor> floors, Vehicle vehicle) {
        for (Floor floor : floors) {
            ParkingSpot spot = floor.findAvailableSpot(vehicle);
            if (spot != null) return spot;
        }
        return null;   // no spot anywhere
    }
}

class NearestFirstStrategy implements SpotSelectionStrategy {
    public ParkingSpot selectSpot(List<Floor> floors, Vehicle vehicle) {
        // floors assumed sorted by nearness to entrance;
        // different picking logic -> its own class
        for (Floor floor : floors) {
            ParkingSpot spot = floor.findAvailableSpot(vehicle);
            if (spot != null) return spot;
        }
        return null;
    }
}
```

Brick 7 — ParkingLot (the Context that ties it together)

The capstone. It **holds both strategies** (injected in the constructor) and runs the two real operations. Notice it never knows *how* spots are picked or priced — it just asks the strategy. Swapping a strategy never changes this class.

ParkingLot.java

```
import java.util.ArrayList;
import java.util.List;

public class ParkingLot {
    private List<Floor> floors;
    private SpotSelectionStrategy spotStrategy;   // pluggable
    private PricingStrategy pricingStrategy;      // pluggable
    private int ticketCounter = 0;

    public ParkingLot(SpotSelectionStrategy spotStrategy,
                      PricingStrategy pricingStrategy) {
        this.floors = new ArrayList<>();
        this.spotStrategy = spotStrategy;         // injected from outside
        this.pricingStrategy = pricingStrategy;
    }

    public void addFloor(Floor floor) { floors.add(floor); }

    // ===== ENTRY =====
    public Ticket parkVehicle(Vehicle vehicle) {
        ParkingSpot spot = spotStrategy.selectSpot(floors, vehicle);

        if (spot == null) {                       // handle LOT FULL
            System.out.println("No spot for " + vehicle.getType());
            return null;
        }

        spot.park(vehicle);
        String ticketId = "T" + (++ticketCounter);
        Ticket ticket = new Ticket(ticketId, vehicle, spot);

        System.out.println("Parked " + vehicle.getLicensePlate()
            + " at " + spot.getId() + ". Ticket: " + ticketId);
        return ticket;
    }

    // ===== EXIT =====
    public double exitVehicle(Ticket ticket) {
        long exitTime = System.currentTimeMillis();
        long durationHours =
            (exitTime - ticket.getEntryTime()) / (1000 * 60 * 60);
        if (durationHours == 0) durationHours = 1;   // min 1 hour

        VehicleType type = ticket.getVehicle().getType();
        double fare = pricingStrategy.calculate(type, durationHours);

        ticket.getSpot().removeVehicle();            // free the spot

        System.out.println("Exited " + ticket.getVehicle().getLicensePlate()
            + ". " + durationHours + "h. Fare: Rs." + fare);
        return fare;
    }
}
```

Min-charge edge case: `if (durationHours == 0) durationHours = 1;` — 20 minutes would integer-divide to 0 hours = free parking. Catching edge cases like this is exactly what impresses in a round.

6. The Key Skill — Data vs Behaviour

The core LLD judgment: **separate classes, or one class with a field?** One question decides it:

> **“Do they DO something different, or just HAVE different values?”**
> DO differently (different logic) → **behaviour** → separate classes.
> HAVE different values (same logic, different numbers) → **data** → one class + field / enum / map.

	Vehicle types	Payment methods
Act differently?	No — all park & exit the same	Yes — each pays differently
What differs?	type & price (values)	the whole process
Subclass body would be...	empty	full of unique logic
Verdict	DATA → enum field	BEHAVIOUR → classes

> **Fast check — the empty-body test:** mentally write the subclass. If its body would be empty or just “return a different number,” it's **data** (use a field). If it'd have genuinely different logic, it's **behaviour** (use a class). This one test drives half your LLD decisions.

Pricing was the tricky mixed case: the **scheme** (hourly vs daily) is behaviour → classes; the **rates** (10/20/40) are data → a lookup map inside. That's why we used `HourlyPricing` with a rate table, not three per-vehicle pricing classes.

7. Full Runnable Code

Every class together, plus a `Main` that builds a lot, parks a car, and exits it. Save each class to its own `.java` file (or all in one for quick testing).

Main.java

```
public class Main {
    public static void main(String[] args) {
        // build the lot with chosen strategies (injected)
        ParkingLot lot = new ParkingLot(
            new FirstAvailableStrategy(),
            new HourlyPricing()
        );

        // set up 1 floor with a few spots
        Floor floor1 = new Floor(1);
        floor1.addSpot(new ParkingSpot("F1-S1", VehicleType.BIKE));
        floor1.addSpot(new ParkingSpot("F1-S2", VehicleType.CAR));
        floor1.addSpot(new ParkingSpot("F1-S3", VehicleType.CAR));
        lot.addFloor(floor1);

        // a car enters
        Vehicle car = new Vehicle("KA01AB1234", VehicleType.CAR);
        Ticket ticket = lot.parkVehicle(car);

        // ...time passes...

        // the car exits
        double fare = lot.exitVehicle(ticket);
    }
}
```

OUTPUT

```
Parked KA01AB1234 at F1-S2. Ticket: T1
Exited KA01AB1234. 1h. Fare: Rs.20.0
```

> **The full class list:** VehicleType (enum), Vehicle, ParkingSpot, Floor, Ticket, PricingStrategy (+ HourlyPricing, FlatDailyPricing), SpotSelectionStrategy (+ FirstAvailableStrategy, NearestFirstStrategy), ParkingLot, Main. All the code for each is in section 5.

8. Patterns & Principles That Emerged

Notice we never *forced* a pattern. Each fell out of good requirements and clean thinking — which is exactly what a machine-coding round tests.

Pattern / Principle	Where it showed up
Strategy (×2)	spot-selection & pricing — pluggable, because we asked “what if the rule changes?”
Data-vs-behaviour	vehicle types = enum (data); selection & pricing schemes = classes (behaviour)
Delegation chain	Lot → Floor → Spot, each doing its own job (Single Responsibility)
Dependency Inversion	the Lot depends on strategy INTERFACES, injected in the constructor
Open/Closed	add a new strategy = new class; existing code untouched

Where are Singleton & Factory?

This problem is often tagged Strategy + Factory + Singleton. We used Strategy twice; the other two are **optional** refinements: the ParkingLot could be a **Singleton** (only one lot exists), and a **Factory** could build vehicles/spots if creation got complex. But we didn't need them for a clean, working design — adding them just to “use more patterns” would be over-engineering.

> **The one lesson to carry into every round:** requirements → entities → build bottom-up → let patterns emerge. Start simple, optimise later, and know when NOT to reach for a pattern. A clean 10-class solution beats a cluttered 40-class one every time.